Serializar y deserializar objetos y binarios puros

Objetivos:

- Trabajar con las clases relacionadas con ficheros de texto, utilizadas en la práctica anterior
- Trabajar con las clases ObjectInputStream y ObjectOutputStrem, específicas para trabajar con los objetos java
- Trabajar con la interfaz Serializable
- Trabajar con las clases DataInputStream y DataOutputStream

EJERCICIO1

En este ejercicio se usa inicialmente el fichero **municipios.csv** que debes tener dentro del proyecto y que contiene los datos de municipios de Cantabria (id del municipio, código, nombre, extensión, habitantes, url del archivo de su bandera, código de comarca y número de localidades que tiene). En el archivo CSV se usa el carácter coma como separador de los datos de un municipio. En la URLs hay el texto "NULL" cuando no tienen esos archivos (con el texto NULL se simboliza el valor null). Y un fichero **comarcas.csv** (código y nombre).

Debes crear una **clase Comarca**, que implemente *la interfaz Serializable* y que contenga los datos código y nombre y una **clase Municipio** que implemente *la interfaz Serializable* para construir objetos que contengan los datos de un municipio excepto los datos id, url de archivo de imagen de bandera y en vez de tener el código de la comarca contenga el objeto Comarca. Añade a esas clases constructores, métodos getter y setter.

Realiza un programa en Java que nada más iniciar el programa cargaremos en una estructura dinámica la información de los objetos municipios, utilizando los ficheros municipios.csv y comarcas.csv.

A continuación, presenta un menú con las siguientes opciones:

1. Añadir municipio al fichero de objetos
2. Listar municipios del fichero de objetos
3. Obtener datos de municipio del fichero de objetos
4. Modificar número de localidades de municipio
5. Eliminar municipio
6. Añadir al fichero de comarcas los municipios
7. Listar el fichero de comarcas.
8. 0- SALIR

Y en función de la opción elegida por teclado realiza cada una de las siguientes operaciones:
Opción 1: Añade al fichero de objetos **municipios.obj** un objeto con los datos de un municipio, cuyo código se ha introducido por teclado, leído de la estructura dinámica elegida.
Hay que comprobar si un código de municipio ya está grabado. Esto debes comprobarlo en un método. Si ya estuviera grabado, se da un mensaje de error y no se añade.

> **NOTA**
> Sólo se debe grabar la cabecera de objetos al grabar el primer objeto del fichero .Sólo el primero se debe grabar usando *ObjectOutputStream*
> Para poder grabar los siguientes, debes usar una clase que debes desarrollar, que *extiende de ObjectOutputStream* y que sobrescribe el **método writeStreamHeader** para que no haga nada.
> Como se trata de añadir al final del fichero los nuevos objetos, debes instanciar el objeto de la *clase FileOutputStream* a partir de la que se instancia el **ObjectOutputStream** que hayas desarrollado, de esta forma:

```
new FileOutputStream(f,true)
```

> Donde **f es el objeto File** que apunta al fichero.
> Si al iniciar el proceso, no existe el fichero, habrá que grabar usando *ObjectOutputStream*. Si, por el contrario, ya existe el fichero, habrá que grabar con la *clase extendida de ObjectOutputStream*.

Opción 2: Se escribirá un listado encolumnado con los datos de los municipios almacenados en el fichero **municipios.obj**.

> **NOTA**
> Cuando se leen objetos de un fichero de objetos mediante **readObject** de *ObjectInputStream*, se produce
> una *EOFException* cuando se ha llegado a fin de fichero (no se puede leer ya un objeto).
> Por ello, la lectura completa de un fichero de objetos se hace con un *bucle while(true)* que estará dentro de un bloque try...catch en el que se controla *EOFException* para detectar el final del fichero.

Opción 3: Se recoge por teclado un código de municipio y se muestran en pantalla los datos del municipio (si existe) almacenado en el fichero **municipios.obj**.

Opción 4: Se pide por teclado el código de un municipio y, si existe en el fichero **municipios.obj**, se escribe en pantalla el nombre del municipio, se pide por teclado el número de localidades de ese municipio y se modifica el contenido de ese municipio en el fichero.

Opción 5: Se pide por teclado el código de un municipio y, si existe en el fichero municipios.obj, se elimina del fichero y se escribe un mensaje indicando el nombre del municipio que se ha eliminado.

Opción 6: Se muestra por pantalla las comarcas existentes y el usuario elige una comarca.
A partir de la estructura dinámica que contiene los objetos Municipios se crea un fichero binario puro **comarcas.dat** en el que se deberá tener la siguiente información para cada municipio de una comarca:

- Cod: Código de la comarca
- Nombre: Nombre de la comarca
- Municipios: array con los nombres de los municipios de la comarca

> **NOTA**
> Ahora vamos a crear un fichero binario puro recuerda que ya no es un objeto y que trabajaremos con *las clases DataInputStream y DataOutputStream*

Opción 7: Se lee el fichero **comarcas.dat** y se muestra su contenido en pantalla.

Opción 8: Salir del programa

Especificaciones

- El fichero de objetos municipios.obj, no contiene internamente una lista de objetos municipios, contiene n- registros de objetos municipios.
- Los métodos por realizar pueden ser todos private static definidos en la Clase de inicio del programa.
- Los métodos void no pueden contener un return.
- Evitar el uso del break.